DataLogger — Monitor de Contaminación Ambiental

Proyecto 2 (Unidad II) | INFO1128 | 2026

Lazarus Pascal · Free Pascal · TLazSerial · TAChart
Flask Python · SQLite · Matplotlib · TFPHTTPClient

Resumen General y Tutorial de Ejecución

Contenido	
1. Descripción General del Proyecto	Pág. 2
2. Arquitectura del Sistema	Pág. 2
3. Componentes Implementados	Pág. 3
4. Estructura de Archivos	Pág. 4
5. Formato de Trama Serial y JSON	Pág. 4
6. API REST — Endpoints Flask	Pág. 5
7. Tutorial: Cómo Arrancar el Proyecto	Pág. 5
8. Compilar el Cliente Lazarus	Pág. 7
9. Resolución de Problemas	Pág. 8

1. Descripción General del Proyecto

Este proyecto implementa un **DataLogger de Contaminación Ambiental** que registra y visualiza en tiempo real los niveles de **Material Particulado (PM1.0, PM2.5 y PM10)** enviados por 5 Unidades Medio Ambientales (UMA1–UMA5) simuladas. La comunicación entre el simulador y el cliente se realiza a través de un **puerto serial virtual (COM1 → COM2 a 9600-8N1)**, y los datos recibidos se grafican en tiempo real y se envían a un **servidor Flask** para su almacenamiento en **SQLite** y visualización en el navegador mediante **Matplotlib**.

El sistema integra tecnologías de **Lazarus Pascal** (cliente de escritorio) con **Python/Flask** (servidor web), demostrando la interoperabilidad entre distintos lenguajes y entornos de desarrollo mediante protocolos estándar (Serial + HTTP/JSON).

2. Arquitectura del Sistema

Componente	Tecnología	Función
Simulador Serial	Python + pyserial	Genera tramas CSV de 5 UMAs y las envía por COM1
Puerto Virtual	VSPE (COM1↔COM2)	Conecta el simulador con el cliente Lazarus
Cliente DataLogger	Lazarus Pascal 4.8 + FPC 3.2.2	Recibe datos seriales, grafica en tiempo real y envía a Flask
Servidor API REST	Python Flask 3.x	Recibe datos JSON, los almacena en SQLite y sirve gráficas
Base de Datos	SQLite (datalogger.db)	Almacena todas las mediciones con fecha y hora
Visualización Web	Matplotlib + HTML/JS	Dashboard con gráficas de series y histograma en el navegador

Flujo de Datos:

3. Componentes Implementados

3.1 Cliente Lazarus Pascal

Componente	Descripción
TLazSerial	Recepción de datos seriales por COM2 (9600-8N1). Evento OnRxData.
TTimer (200 ms)	Actualiza las gráficas de forma segura en el hilo principal.
TChart + TLineSeries × 3	Grafica las series de tiempo de PM1.0, PM2.5 y PM10 (12 muestras, scroll left).
TChart + TBarSeries	Histograma de 6 tamaños de partículas (0.3 → 10 µm) con colores distintos.
TComboBox UMA1-5	Permite seleccionar cuál UMA se desea monitorear y graficar.
TFPHTTPClient	Envía los datos en formato JSON al servidor Flask mediante HTTP POST.
TJSONObject	Construye el objeto JSON con los 12 campos de la trama para enviar al servidor.
TMemo (Log)	Registro de eventos en pantalla: tramas recibidas, errores Flask, conexiones.

3.2 Servidor Flask Python

Módulo / Librería	Función
Flask 3.x	Framework web para API REST (POST/GET)
SQLite3	Base de datos local. Tabla "mediciones" con DateTime automático.
Matplotlib (Agg)	Genera gráficas PNG de series temporales e histograma en memoria.
io.BytesIO	Sirve las gráficas Matplotlib directamente al navegador sin archivos temporales.
json / request	Parsea los datos JSON recibidos por POST del cliente Lazarus.

4. Estructura de Archivos del Proyecto

■ Árbol de directorios del proyecto

```
Proyecto2_interfaces_graficas/
■■■ lazarus_client/
■   ■■■ Lazarus_client.lpi     ← Archivo de proyecto Lazarus
■   ■■■ Lazarus_client.lpr     ← Programa principal
■   ■■■ unit1.pas              ← Código fuente completo (lógica)
■   ■■■ unit1.lfm              ← Diseño de la interfaz gráfica
■   ■■■ Lazarus_client.exe     ← Ejecutable compilado (45 MB)
■■■ serial_sim/
■   ■■■ simu_trama.py          ← Simulador de sensor PMS7003ST
■   ■■■ server_flask.py        ← Servidor Flask completo
■   ■■■ datalogger.db          ← Base de datos SQLite (se crea al iniciar)
■■■ TLazSerial/                ← Librería TLazSerial (GitHub)
■■■ proyecto2.vspe             ← Config VSPE: Par COM1 ↔ COM2
■■■ README.md                  ← Instrucciones de uso
```

5. Formato de Trama Serial y JSON

5.1 Trama Serial (CSV)

El simulador envía una línea de texto por el puerto serial COM1 a 9600-8N1 con los valores separados por comas, terminando en \n:

■ Formato CSV

```
id,mp01,mp25,mp10,h03,h05,h10,h25,h50,h100,te,hr\n

Ejemplo:  1,45,67,80,1200,800,400,200,100,50,22,65
```

Campo	Descripción	Rango
id	Identificador de la UMA (Unidad Medio Ambiental)	1 – 5
mp01	Material Particulado PM1.0 (µg/m³)	5 – 100
mp25	Material Particulado PM2.5 (µg/m³)	5 – 100
mp10	Material Particulado PM10 (µg/m³)	5 – 100
h03	Histograma: partículas de 0.3 µm (conteo)	10 – 10000
h05	Histograma: partículas de 0.5 µm (conteo)	10 – 10000

h10	Histograma: partículas de 1.0 µm (conteo)	10 – 10000
h25	Histograma: partículas de 2.5 µm (conteo)	10 – 10000
h50	Histograma: partículas de 5.0 µm (conteo)	10 – 10000
h100	Histograma: partículas de 10 µm (conteo)	10 – 10000
te	Temperatura del ambiente (°C)	5 – 35
hr	Humedad Relativa del aire (%)	20 – 80

5.2 JSON enviado a Flask

```
■ Body JSON (Content-Type: application/json)
{
  "id":   1,
  "mp01": 45,   "mp25": 67,   "mp10": 80,
  "h03":  1200, "h05":  800,  "h10":  400,
  "h25":  200,  "h50":  100,  "h100": 50,
  "te":   22,   "hr":   65
}
```

6. API REST — Endpoints Flask

Método	URL	Descripción
GET	/	Página de inicio con listado de endpoints
GET	/dashboard	Dashboard visual con gráficas (auto-refresh 10s)
POST	/datos	Recibir datos JSON, almacenar en SQLite con DateTime
GET	/datos	Listar todas las mediciones en formato JSON
GET	/datos/uma/<id>	Filtrar mediciones por UMA (1-5) en JSON
GET	/datos/ultimas/<n>	Obtener las últimas N mediciones en JSON
GET	/estadisticas	Promedios por UMA (PM1, PM2.5, PM10, T°, HR)
GET	/grafica	Imagen PNG con series de tiempo de todas las UMAs
GET	/grafica/uma/<id>	Imagen PNG con series de tiempo de una UMA específica
GET	/grafica/histograma	Imagen PNG con histograma de partículas (promedio)

7. Tutorial: Cómo Arrancar el Proyecto

*Sigue los pasos en **el orden indicado**. Cada componente depende del anterior.*

PASO 1 — Activar Virtual Serial Port Emulator (VSPE)

VSPE crea un par de puertos seriales virtuales conectados entre sí: lo que se escribe en **COM1** es leído por **COM2** y viceversa.

- Abrir la aplicación **VSPE** (Virtual Serial Port Emulator).
- Ir a **Device** → **Create** → seleccionar **Connector (Pair)**.
- Configurar: **COM1** y **COM2** → clic en **Create**.
- También puedes cargar el archivo **proyecto2.vspe** desde *File* → *Open*.
- Asegurarse de que el par quede activo (ícono verde).

■ Si no tienes VSPE, puedes instalar com0com (open source) desde https://sourceforge.net/projects/com0com/

PASO 2 — Iniciar el Servidor Flask

El servidor Flask gestiona la base de datos y la API. Debe estar corriendo **antes** de intentar enviar datos desde Lazarus.

Abrir una terminal **PowerShell** o **CMD** y ejecutar:

■ Iniciar servidor Flask

```
# Navegar al directorio del simulador
cd "c:\Users\Kuro_Waza\.antigravity-ide\Proyecto2_interfaces_graficas\serial_sim"

# Iniciar el servidor
python server_flask.py

# Salida esperada:
# ============================================================
#   DataLogger Flask Server
#   http://127.0.0.1:5000
#   http://127.0.0.1:5000/dashboard
# ============================================================
# [DB] Base de datos lista en: ...datalogger.db
#  * Running on all addresses (0.0.0.0)
#  * Running on http://127.0.0.1:5000
```

*✔ Verificar abriendo el navegador en **http://127.0.0.1:5000** — debe mostrar la página de inicio.*

PASO 3 — Iniciar el Simulador Serial

El simulador genera datos aleatorios de los 5 sensores UMA y los envía por **COM1** al ritmo de una trama cada 100 ms.

Abrir **otra terminal** y ejecutar:

■ Iniciar simulador de trama serial

```
# Navegar al directorio
cd "c:\Users\Kuro_Waza\.antigravity-ide\Proyecto2_interfaces_graficas\serial_sim"

# Ejecutar con puerto COM1, intervalo 0.1 segundos
python simu_trama.py COM1 0.1
```

```
# Salida esperada (una trama por linea):
# [000000] 1,45,67,80,1200,800,400,200,100,50,22,65
# [000001] 2,38,71,55,950,720,350,180,90,45,19,58
# [000002] 3,61,49,92,1100,680,420,210,110,55,25,71
# ...

# Para cambiar el intervalo a 1 segundo:
python simu_trama.py COM1 1.0

# Para detener: Ctrl + C
```

PASO 4 — Ejecutar el Cliente Lazarus (DataLogger)

El cliente Lazarus es la interfaz gráfica principal que muestra las gráficas en tiempo real y permite enviar datos al servidor Flask.

```
■ Ejecutar el cliente (doble clic o desde terminal)
# Opción A: Doble clic en el explorador de archivos
c:\Users\Kuro_Waza\.antigravity-ide\Proyecto2_interfaces_graficas\
lazarus_client\Lazarus_client.exe

# Opción B: Desde PowerShell
Start-Process ".\lazarus_client\Lazarus_client.exe"
```

Una vez abierta la aplicación:

a) En el campo **Puerto COM**, escribir **COM2** (sin comillas).

b) En **Sensor UMA**, seleccionar la UMA que deseas monitorear (UMA1-UMA5).

c) Si deseas almacenar los datos en Flask/SQLite, activar el checkbox **"Enviar a Flask"**.

d) Verificar que la URL Flask sea **http://127.0.0.1:5000/datos**.

e) Clic en el botón **Conectar** → la barra de estado debe mostrar "Conectado a COM2" en verde.

f) Las gráficas comenzarán a actualizarse automáticamente cada 200 ms.

> ✔ *Los datos de temperatura y humedad se muestran en la barra superior. El log inferior muestra cada trama recibida y el resultado del envío a Flask.*

PASO 5 — Ver el Dashboard en el Navegador

El dashboard muestra gráficas generadas por Matplotlib directamente en el navegador, con auto-actualización cada 10 segundos.

```
■ Abrir el dashboard
# Abrir en el navegador:
http://127.0.0.1:5000/dashboard

# Dashboard para una UMA específica:
http://127.0.0.1:5000/dashboard?uma=1

# Ver datos JSON de todas las UMAs:
http://127.0.0.1:5000/datos

# Ver estadísticas por UMA:
http://127.0.0.1:5000/estadisticas
```

```
# Ver gráfica de UMA2:
http://127.0.0.1:5000/grafica/uma/2

# Ver histograma de partículas:
http://127.0.0.1:5000/grafica/histograma
```

8. Compilar el Cliente Lazarus (si se requiere)

El ejecutable Lazarus_client.exe ya está compilado. Si necesitas recompilar (por ejemplo, tras modificar el código fuente), usa los siguientes métodos:

8.1 Compilar desde PowerShell (lazbuild)

```
■ Compilar con lazbuild desde PowerShell
# Agregar Lazarus y FPC al PATH
$env:PATH = "C:\lazarus;C:\lazarus\fpc\3.2.2\bin\x86_64-win64;" + $env:PATH
$pcp = "c:\Users\Kuro_Waza\.lazarus"

# Compilar el paquete TLazSerial (solo la primera vez)
& "C:\lazarus\lazbuild.exe" --pcp=$pcp `
  ".\TLazSerial\LazSerialPort.lpk"

# Compilar el proyecto principal
& "C:\lazarus\lazbuild.exe" --pcp=$pcp `
  ".\lazarus_client\Lazarus_client.lpi"

# Salida esperada al finalizar:
# (1008) 436 lines compiled, 18.4 sec
```

8.2 Compilar desde el IDE Lazarus (recomendado para desarrollo)

1. Abrir **C:\lazarus\lazarus.exe**.
2. Ir a **File** → **Open Project** → seleccionar **lazarus_client\Lazarus_client.lpi**.
3. Si TLazSerial no está instalado: ir a **Package** → **Open Package File (.lpk)** → abrir **TLazSerial\LazSerialPort.lpk** → clic en **Compile** → **Install**.
4. Ir a **Run** → **Build** (o presionar **F9**) para compilar y ejecutar.
5. El ejecutable se genera en **lazarus_client\Lazarus_client.exe**.

Paquetes requeridos:

Paquete	Descripción	Ubicación
LCL	Lazarus Component Library (incluido)	C:\lazarus\lcl
tachartlazaruspkg	TAChart - gráficas (incluido en Lazarus)	C:\lazarus\components\tachart
LazSerialPort	TLazSerial - comunicación serial (descargado)	TLazSerial\LazSerialPort.lpk
fphttpclient	Cliente HTTP (incluido en FPC/FCL)	C:\lazarus\fpc\3.2.2\...
fpjson / jsonparser	Manejo de JSON (incluido en FPC)	C:\lazarus\fpc\3.2.2\...

9. Resolución de Problemas Comunes

Problema	Causa probable	Solución
No se grafican datos en Lazarus	COM2 no recibe datos	Verificar que VSPE está activo con COM1↔COM2, y que el simulado
"Error al conectar" en Lazarus	COM2 no existe o está ocupado	Verificar VSPE. Abrir Administrador de dispositivos → Puertos (COM y
Flask responde HTTP 500	Error en el JSON enviado	Verificar que Content-Type sea application/json. Ver log en la ventana
Las gráficas no se muestran en el navegador	Sin datos en la BD o Matplotlib falla	Enviar al menos una medición. Revisar que matplotlib está instalado: p

El simulador da "puerto no existe"	VSPE no está activo o COM1 no está configurado	Activar VSPE primero. Si hay error de acceso, ejecutar como Administ
Error al compilar "Identifier not found"	Falta el paquete en el proyecto	Abrir Lazarus IDE → instalar TLazSerial y agregar tachartlazaruspkg a
Flask: "Address already in use" (puerto 5000)	Otro proceso usa el puerto 5000	Cerrar otros servidores Flask. O cambiar puerto en server_flask.py (ap

---